

PROJECT REPORT

FPP

Faculty Patent Verification Portal — Project Report

Author: Mohan Prabu K — B.Tech Artificial Intelligence & Data Science,
R P Sarathy Institute of Technology

Repository: <https://github.com/MohanPrabu018-K/Faculty-Patent-Collection-and-Verification-Portal>

Live demo: <https://faculty-patent-portal.vercel.app>

Document date: October 2026 | Version 1.0

Status: Currently Building

1. Executive Summary

FPP (Faculty Patent Verification Portal) is an institutional web portal that collects, verifies, organizes, and manages faculty patents and design registrations. Its guiding principle is: AI assists, rules validate, evidence supports, and humans confirm uncertainty — with PostgreSQL as the single source of truth. The system runs a deterministic 11-step document pipeline (classification, QR, OCR extraction, understanding, verification, identity, duplicate, conflict, association, data quality, analytics) behind a FastAPI backend and a React + TypeScript frontend.

2. Problem Statement

Institutions lack a structured way to gather faculty IP records, verify their authenticity, detect duplicates and conflicts, and keep a reviewable evidence trail. Manual collection over email and spreadsheets is error-prone and unverifiable. FPP provides an upload-to-verified-review workflow with machine extraction plus rule-based validation and human confirmation of uncertain cases.

3. Objectives

- Accept patent/design document uploads with tracked processing jobs and status polling.
- Extract text via OCR with graceful degradation when keys or APIs are unavailable.
- Verify identity, detect duplicates and conflicts, and associate records to faculty profiles.
- Route uncertain cases to structured human review with persisted evidence.

4. System Architecture

FastAPI + Pydantic v2 (async) API layer with SQLAlchemy 2.0 + Alembic over PostgreSQL 16. Document processing runs deterministically in-process via FastAPI BackgroundTasks — no Celery, no Redis, no queue. PDFs with embedded text skip OCR entirely; scanned pages go through the local engine chain (PaddleOCR, then Tesseract) with deterministic preprocessing. The React 18 + TypeScript + Vite frontend polls upload status until a terminal state is reached.

Layer	Implementation
API	FastAPI (async), Pydantic v2, SQLAlchemy 2.0 + Alembic migrations
Database	PostgreSQL 16 — single source of truth
Pipeline	11 deterministic agents executed via BackgroundTasks (in-process)
OCR / docs	Local-first chain verified in code: PaddleOCR (optional) then Tesseract local via pytesseract (guaranteed engine, confidence-scored), with OCR.Space as opt-in emergency fallback; preprocessing variants (grayscale, autocontrast, Otsu threshold, 2x upscale); embedded-text fast path via PyMuPDF
Frontend	React 18 + TypeScript + Vite
Auth	Argon2id hashing + JWT httpOnly SameSite cookie + double-submit CSRF

5. Key Features

- Document upload producing IpRecord / IpFile / ProcessingJob rows with QUEUED status and 202 Accepted responses.
- 11-step pipeline: Classification, QR, OCR Extraction, Understanding, Verification, Identity, Duplicate, Conflict, Association, Data Quality, Analytics.

- Terminal processing states: COMPLETED, COMPLETED_WITH_ERRORS, AWAITING_REVIEW, FAILED — with pollable status endpoint.
- Duplicate and conflict checks with evidence-backed structured review.

6. Data Flow

- POST /api/v1/uploads/ → records created → background task runs the 11-step pipeline.
- Agents persist extracted fields and job rows to PostgreSQL.
- GET /api/v1/uploads/{id}/status → UI polls until a terminal status, then review.

7. Implementation Highlights

- Free-first design: no Docker, no mandatory paid APIs; Tesseract runs fully local with confidence scoring, and the external OCR fallback stays disabled unless explicitly configured.
- QR decoding tries pyzbar across grayscale, rotated, and upscaled variants with an OpenCV fallback, including IP-India URL recognition.
- Native run without containers: Python 3.12 backend (uvicorn) plus Node 20 frontend; PostgreSQL 16; Alembic migrations; pytest unit/contract/agent tests plus a real end-to-end background-task pipeline script.

8. Testing & Verification

- Backend pytest suite (unit / contract / agent tests); real upload-to-PostgreSQL end-to-end pipeline script (needs live DB); frontend production build and browser end-to-end tests where configured.

9. Results & Current Status

The repository demonstrates a working upload-to-verified-review architecture with container-free local execution, Docker and Render deployment descriptors, and full project documentation (blueprint, audit report, remaining-work tracker). Active development continues toward completion, consistent with its Currently Building status in the portfolio.

10. Challenges & Future Scope

- Challenge: reliable OCR without paid services — addressed with OCR.Space free tier, retry/backoff, and graceful degradation.
- Future: items tracked in the repository REMAINING_WORK documentation; hardening review workflows and deployment.

11. Repository & Reproduction

Repository: <https://github.com/MohanPrabu018-K/Faculty-Patent-Collection-and-Verification-Portal> — backend via uvicorn with Alembic migrations; frontend via npm install and npm run dev. Live demo: <https://faculty-patent-portal.vercel.app>. OCR configuration uses a free OCR.Space key set locally and never committed.